Discovery Executive Summary

Project: discovery- Dev Env · **Generated:** 7/16/2026, 11:54:10 AM

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 1 discovery analysis run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating	Hotspot Score
1	Security Analysis	HIGH RISK	—

1. Security Analysis

OVERALL CODEBASE RATING — SECURITY

High Risk

Driven by DOM XSS in InputFilter, 8 critical/high npm audit findings in direct runtime deps, client-side-only auth guards, and JWT/session tokens in browser storage.

EXECUTIVE SUMMARY

The target workspace contains two React frontends with no server-side application source; API calls target an external backend at `localhost:3030/api/`. **Backend:** not present in scope — SQL injection, CSRF enforcement, and CORS policy cannot be verified here. **Frontend:** both apps were reviewed (FS1–FS5). The most severe finding is DOM-based XSS in `social-media-react` where autocomplete suggestions are built with unsanitized `innerHTML` from user input. Additional high-risk issues include JWT/session data in browser storage (`localStorage` / `sessionStorage`), client-side-only route guards, a hardcoded Google Maps API key shipped in the bundle, and unvalidated `href` values from API post data. `npm audit` reports **6 critical** and **34 high** vulnerabilities in `social-media-react` (75 total) and **2 critical** and **19 high** in `workbench-demo` (40 total), including outdated direct dependencies `axios@0.27.2` and `react-scripts@5.0.1`. No Content-Security-Policy headers, no dependency-scan CI step, and no CSRF token handling were observed in client code.

6.1 Security Benchmark Ratings

#	Security KPI	Target	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Critical Vulnerabilities	0	0	1	>1	7 (1 DOM XSS + 6 npm critical in social-media-react)	HIGH RISK
H2	High Vulnerabilities	0	<5	5–10	>10	57 (8 code/auth findings + 34+19 npm high)	HIGH RISK
H3	Medium Vulnerabilities	low	<20	20–50	>50	58 (5 code findings + 20+29+4 npm moderate/low runtime-adjacent)	HIGH RISK
H4	Vulnerability Density	<0.5/KLOC	<0.5	0.5–1.0	>1.0	1.45/KLOC (10 findings / 6.9 KLOC)	HIGH RISK
H5	OWASP Top 10 Compliance	>95%	>95%	80–95%	<80%	10% clean (1/10 applicable categories)	HIGH RISK
H6	Critical/High Vulnerable Deps	0	0	1	>1	61 (6+2 critical, 34+19 high across both apps)	HIGH RISK
H7	Outdated Dependencies	<10%	<10%	10–25%	>25%	~35% (axios 0.27.2, react-router-dom 5.3.0, react-scripts 5.0.1, CRA stack)	HIGH RISK
H8	End-of-Life Dependencies	0	0	1–5	>5	3 (axios 0.27.x, react-scripts/CRA 5.x, react-router-dom 5.x)	MODERATE

6.5 Actions Required

Finding	Action	Rating	Priority
DOM XSS via innerHTML (`InputFilter.jsx`)	Replace `innerHTML` with React-rendered list items; add CSP `script-src 'self'`	HIGH RISK	CRITICAL
Vulnerable npm dependencies (axios, react-scripts, transitive)	Upgrade axios to ≥1.15.2 in both apps; run `npm audit fix` ; add CI audit gate	HIGH RISK	CRITICAL
Hardcoded Google Maps API key (`Map.jsx`)	Rotate key; move to `REACT_APP_*` env with referrer restrictions	HIGH RISK	HIGH
JWT/session in browser storage	Migrate to `httpOnly` cookie sessions; remove `localStorage` / `sessionStorage` token storage	HIGH RISK	HIGH
Client-side-only authorization	Add server session validation on bootstrap; protect `/dashboard` and `/main` routes server-side	HIGH RISK	HIGH

Missing Content-Security-Policy	Add CSP via meta tag or reverse-proxy response headers on both apps	HIGH RISK	MEDIUM
Credentialed requests without CSRF tokens	Implement CSRF double-submit token; set `SameSite=Strict` cookies on backend	HIGH RISK	MEDIUM
Unsafe API link href (`PostBody.jsx`)	Allow-list `https://` / `http://` schemes; reject `javascript:` URLs server-side	HIGH RISK	MEDIUM
Unvalidated file upload (`imgUpload.service.js`)	Add MIME type and size limits client-side; enforce magic-byte validation server-side	HIGH RISK	MEDIUM
No dependency-scan CI (DevSecOps)	Add `npm audit --audit-level=high` and optional SAST to CI pipeline	HIGH RISK	MEDIUM
Verbose client error logging	Remove `console.dir(err)` from production builds; redact error responses	MODERATE	LOW

Full report saved to `target/docs/discovery/06-security.md` (pipeline copy at `agent-runs/20260716T114742_k6rmmk/06-security.md`). The orchestration UI will convert it to PDF automatically.